

## Project Initialization and Planning – Proposed Solution

**Date:** 30 September 2026      **Team ID:** 6ab3e57da238dc999eb3d44d      **Project Name:** EduGenie – AI-Powered Educational Assistant

**Maximum Marks: 5 Marks**

## Project Overview

EduGenie is a modular AI-powered educational assistant that gives learners five focused study workflows through a single browser interface. The application uses FastAPI for the backend, a lightweight HTML/CSS/JavaScript frontend, and a shared AI service that can connect to Google Gemini. A demo fallback keeps the application usable for local development when a live API key is unavailable.

## Objective

To reduce friction in self-learning by turning questions, topics and study passages into understandable answers, explanations, summaries, practice quizzes and structured learning recommendations.

## Scope

The current scope covers Q&A, concept explanation, summarization, three-question MCQ generation, and beginner-to-advanced learning recommendations. Authentication, persistent history, teacher dashboards and document upload are outside the current release.

## Problem Statement

Learners often use several disconnected tools to understand a topic, revise notes, practice questions and decide what to learn next. This creates extra effort and can make learning less structured.

## Proposed Approach

| Component | Approach                                                                                           |
|-----------|----------------------------------------------------------------------------------------------------|
| Frontend  | Single web workspace with task selection, input area and result display.                           |
| Backend   | FastAPI routes requests to focused feature modules.                                                |
| AI Layer  | Shared AIService centralizes Gemini configuration, live generation and demo fallback.              |
| Quality   | Pydantic input validation, quiz JSON cleaning/parsing and explicit error handling.                 |
| Future    | Persistent progress, PDF/image input, multilingual support, authentication and deployment scaling. |

## Resource Requirements

Development can run on a normal Python-capable computer. A live Gemini API key is optional for local demo mode; a production deployment would require secure secret management, monitoring and appropriate cloud resources.